UNIVERSITY OF KARACHI

UBIT — Department of Computer Science

PROJECT PROPOSAL

MULTIPURPOSE TOOL APPLICATION

Java Console-Based Utility Application

Course	Object Oriented Programming (SE-353)
Instructor	Ms. Ilsa Naeem
Project	Multipurpose Tool Application
Group Leader	Mohsin Ali — Roll No. 23
Group Member	Jawad Ahmed — Roll No. 21
Group Member	Ali Subhan — Roll No. 6
Group Member	M. Zaid — Roll No. 54
Project Type	Java Console Application

1. Project Overview

The Multipurpose Tool Application is a Java-based console application that combines several practical calculation and conversion utilities into one unified platform. The application is designed to provide users with a simple and organized way to perform common calculations without switching between separate tools.

The system follows a modular and object-oriented structure. A central MainApp controls navigation while individual Java classes handle the specific tools. The improved version also provides a cleaner console interface, input validation, error handling, calculation history, and an About Project section.

2. Problem Statement

Users often need different applications or calculators for routine tasks such as arithmetic calculations, distance conversions, age calculations, fuel estimation, and currency conversion. Switching between separate tools can be inconvenient and time-consuming. The Multipurpose Tool Application addresses this problem by combining these utilities into one menu-driven Java application.

3. Aim of the Project

The main aim of the project is to develop a user-friendly, modular, and object-oriented Java application that provides multiple everyday calculation and conversion tools through a single console interface. The project also provides practical experience in object-oriented programming concepts, classes, objects, methods, input handling, validation, and program organization.

4. Objectives

- Combine multiple everyday calculation and conversion tools into one application.
- Apply object-oriented programming concepts through separate classes and objects.
- Provide a clear and user-friendly console interface.
- Implement reliable input validation and error handling.
- Maintain a reusable and modular code structure.
- Provide calculation history for operations performed during a session.
- Demonstrate practical use of Java control structures, methods, classes, objects, and input handling.
- Create a foundation that can be extended with additional tools and a graphical interface in the future.

5. Main Application Modules

Module	Purpose	Main Functions
1. Fuel Commute Estimator	Estimates fuel requirement and travel cost.	Fuel type selection, vehicle/engine options, fuel average, distance, liters required, and total cost.
2. Age Calculator	Calculates age using date information.	Birth date input, current date handling, and age calculation.
3. Currency Converter	Converts between supported currencies.	Currency selection, amount input, conversion calculation, and formatted result.
4. Distance Converter	Converts between common distance units.	Kilometers, miles, meters, and feet conversions.
5. Basic Calculator	Performs common mathematical	Addition, subtraction, multiplication,

	operations.	division, and modulus.
6. Calculation History	Keeps a record of calculations during the current session.	Stores and displays completed calculation results.
7. About Project	Displays project and group information.	Project title, course, and team information.

6. Scope of the Project

The project covers a Java console application for educational and personal use. It includes five core calculation/conversion modules supported by common navigation, validation, error handling, calculation history, and project information. The application is designed to work locally without requiring internet connectivity for its core operations.

- Console-based interaction.
- Five core utility modules.
- Central menu-driven navigation.
- Reusable utility and interface functions.
- Input validation and error messages.
- Session-based calculation history.
- About Project information.

7. Functional Requirements

- The system shall display a main menu containing the available tools and an Exit option.
- The system shall allow users to select a tool using a menu choice.
- The Fuel Commute Estimator shall calculate estimated fuel consumption and cost from valid inputs.
- The Age Calculator shall calculate age from date information.
- The Currency Converter shall perform conversions using the configured currency rates.
- The Distance Converter shall convert between supported distance units.
- The Basic Calculator shall perform addition, subtraction, multiplication, division, and modulus.
- The system shall reject invalid numeric inputs and request valid input where required.
- The system shall prevent division and modulus by zero.
- The system shall allow the user to return to the main menu after completing a tool.
- The system shall store completed calculations in session-based history.
- The system shall provide an About Project option containing project and group information.
- The system shall exit safely when the user selects the Exit option.

8. Non-Functional Requirements

- Usability — Menus, prompts, and results should be clear and understandable.
- Reliability — Valid inputs should produce consistent results.
- Maintainability — Individual tools should remain separated into manageable classes.
- Efficiency — Calculations should execute quickly with normal console input.
- Portability — The application should run on systems with a compatible Java runtime.
- Offline Operation — Core tools should not require an internet connection.

9. Object-Oriented Design

The application is organized around Java classes representing separate tools and supporting utilities. MainApp acts as the central coordinator and creates or uses the required module objects. Each calculator class contains the logic for its own responsibility, while common console and history functionality is kept separate for reuse.

- Classes — Separate classes represent the individual tools.
- Objects — MainApp uses objects to access calculator functionality.
- Encapsulation — Calculation logic is kept within the relevant classes.
- Modularity — Each tool can be maintained independently.
- Reusability — Shared input and console functions reduce repeated code.

10. System Architecture

Component	Responsibility	Role
MainApp	Controls application flow and navigation.	Main menu and module coordination
FuelCalculator	Calculates fuel requirement and commute cost.	Fuel utility
AgeCalculator	Calculates age from date information.	Age utility
CurrencyConverter	Performs currency conversions.	Currency utility
DistanceConverter	Performs distance conversions.	Distance utility
SimpleCalculator	Performs arithmetic operations.	Arithmetic utility
ConsoleUI	Provides reusable console formatting and input handling.	User interface support
CalculationHistory	Stores and displays completed calculations.	History support

11. User Interface and Navigation

The application uses a structured console interface with welcome and exit screens, clearly separated menus, status messages, input prompts, and error messages. Each tool provides a clear path back to the main menu. This design improves usability while keeping the project appropriate for a Java console-based academic application.

12. Processing Flow

The general application flow is: Start → Welcome Screen → Main Menu → Select Tool → Enter Input → Validate Input → Perform Calculation/Conversion → Display Result → Save Result to History → Return to Main Menu → Exit.

- Input: The user selects a tool and enters the required values.
- Validation: The application checks the input before processing.
- Processing: The selected module performs the required calculation.
- Output: The result is displayed clearly to the user.
- History: The completed calculation can be retained in the current session.
- Navigation: The user returns to the main menu or exits the application.

13. Testing and Validation Plan

Testing will be performed at both module and application levels. The upgraded version places particular emphasis on invalid input handling and edge cases.

- Unit Testing — Test individual calculation methods with known inputs and expected outputs.
- Normal Input Testing — Verify typical calculations and conversions.
- Boundary Testing — Test zero, small, large, and other meaningful boundary values.
- Invalid Input Testing — Enter letters, unsupported choices, negative values where inappropriate, and malformed input.
- Arithmetic Error Testing — Confirm division and modulus by zero are handled safely.
- Integration Testing — Confirm that MainApp correctly connects all modules and returns to the main menu.
- User Testing — Run complete sessions to verify navigation, results, history, and exit behavior.

14. Expected Outcomes

- A complete Java-based Multipurpose Tool Application.
- Practical demonstration of object-oriented programming concepts.
- A cleaner and more user-friendly console interface.
- Reliable handling of valid and invalid user input.
- Successful integration of five practical utility modules.
- Session-based calculation history for improved usability.
- A modular foundation for future development.

15. Future Enhancements

- Graphical User Interface (GUI) to replace the console interface.
- Database integration for permanent calculation history and preferences.
- Live currency exchange rates through an API.
- User accounts and personalized profiles.
- PDF or Excel report export.
- Additional calculators and conversion tools.

16. Group Members

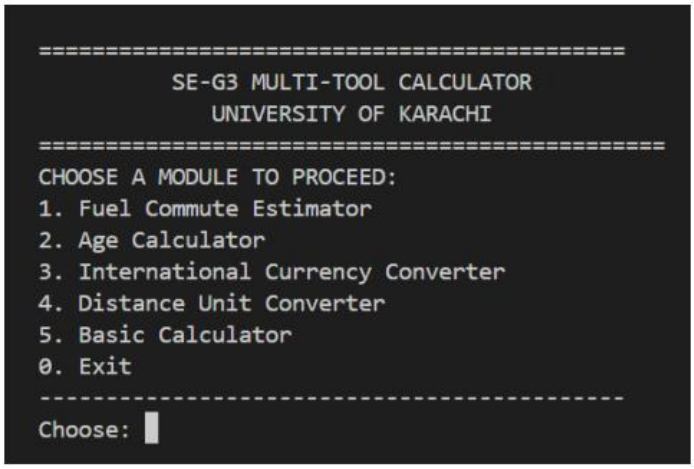
Name	Roll No.	Role
Mohsin Ali	23	Group Leader
Jawad Ahmed	21	Member
Ali Subhan	6	Member
M. Zaid	54	Member

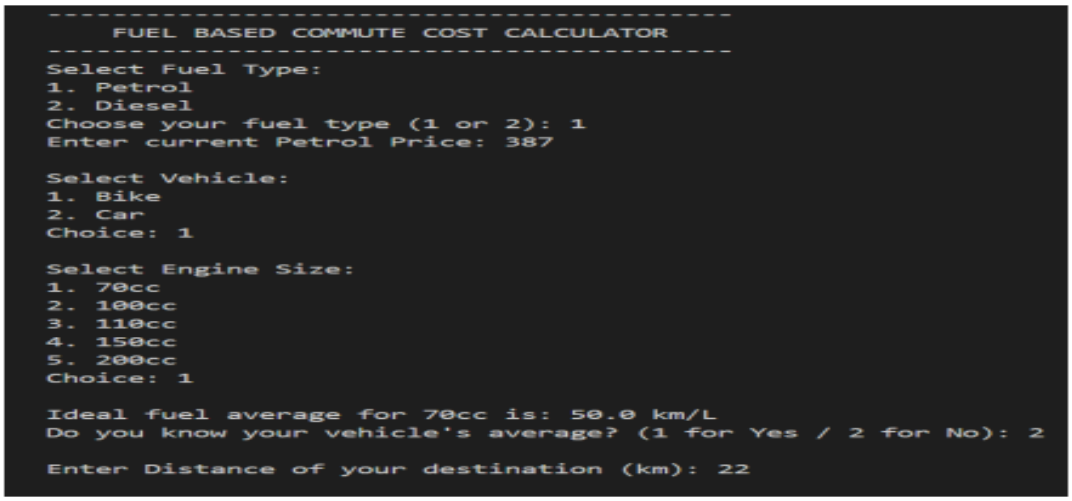
17. Conclusion

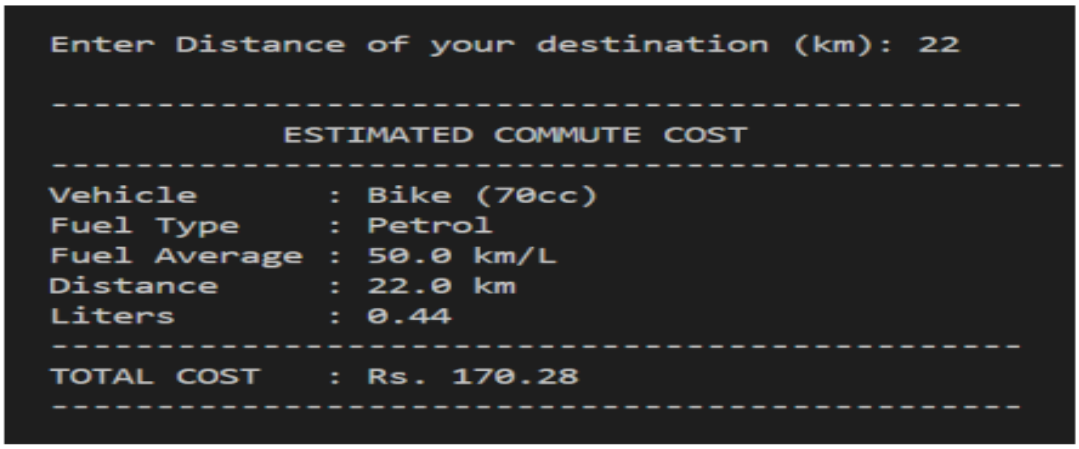
The Multipurpose Tool Application is a practical Java project that integrates several everyday utilities into one organized platform. Its modular object-oriented design separates the responsibilities of each tool while MainApp provides centralized navigation. The improved version strengthens the original application through better console presentation, input validation, error handling, calculation history, and reusable utility classes.

The project provides a strong practical demonstration of Object Oriented Programming concepts and can be further expanded into a graphical or database-backed application.

18. Screen Shots:







```
-----------------------------------
          AGE CALCULATOR
-----------------------------------
STEP 1: Enter Current Date
Current Year (YYYY): 2026
Current Month (1-12): 5
Current Day (1-31): 21

STEP 2: Enter Birth Date
Birth Year (YYYY): 2006
Birth Month (1-12): 8
Birth Day (1-31): 19

-----------------------------------
      AGE CALCULATION RESULT
-----------------------------------
Current Date : 21/5/2026
Birth Date   : 19/8/2006
-----------------------------------
YOUR TOTAL AGE: 19 Years
-----------------------------------
```

```
-----------------------------------
      CURRENCY CONVERTER (G3)
-----------------------------------
Select Direction:
1. PKR to Foreign Currency
2. Foreign Currency to PKR
Choice: 2

Select Currency:
1. US Dollar (USD)
2. Euro (EUR)
3. Chinese Yuan (CNY)
4. UAE Dirham (AED)
Choice: 1
Enter Amount: 4300
Result: Rs. 1204000.00
-----------------------------------
```

```
-----------------------------------
      DISTANCE CONVERTER (G3)
-----------------------------------
Convert FROM:
1. Kilometers (km)
2. Miles (mi)
3. Meters (m)
4. Feet (ft)
Choice: 1
Enter Distance: 321

Convert TO:
1. Kilometers (km)
2. Miles (mi)
3. Meters (m)
4. Feet (ft)
Choice: 3
-----------------------------------
RESULT: 321000.0000
-----------------------------------
```

```
-----------------------------------
      BASIC MATH CALCULATOR (G3)
-----------------------------------
Enter First Number: 32
Enter Second Number: 123

Select Operation:
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Modulas (%)
Choice: 3
-----------------------------------
Result: 3936.0
-----------------------------------
```